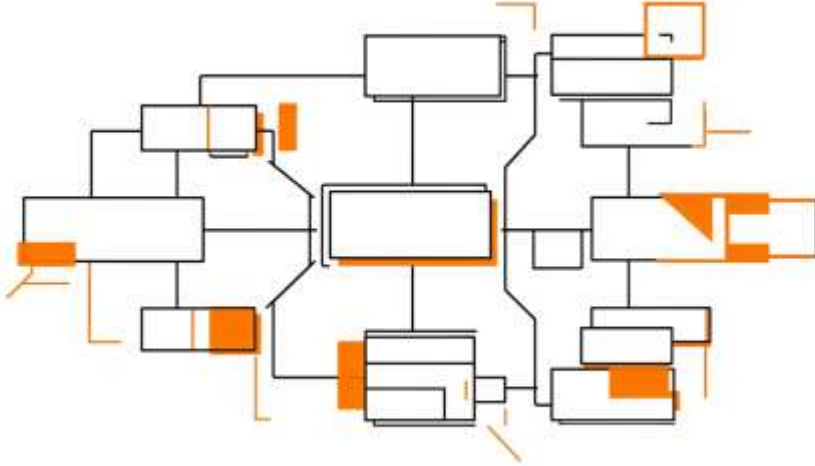


Unlock the Power of Databases: Free RDBMS Books for Your Website

Comprehensive resources to help your audience master relational
database systems



The Foundation of Modern Data: Why RDBMS Matters



Relational Database Management Systems (RDBMS) form the backbone of virtually every modern information system, from banking platforms to social networks to enterprise applications.

Understanding RDBMS principles is essential for developers building scalable applications, data analysts extracting insights from complex datasets, and IT professionals managing enterprise infrastructure.

Access to Knowledge: Free RDBMS Books Available Online



Quality Content

Leading textbook authors and publishers now offer complete RDBMS guides in accessible PDF format



Free Access

Many authoritative texts are available at no cost for personal and educational use



Easy Integration

Direct links to PDFs make it simple to add these resources to your website instantly

This guide curates the most valuable free RDBMS textbooks, complete with direct access links and key features for each resource.

Database Systems: Concepts, Languages and Architectures

Atzeni et al., McGraw Hill

This comprehensive textbook, now out of print and available for free, provides a modular and progressive approach to database management systems. It covers essential topics from foundational querying to advanced schema design.

Key Coverage:

- Database querying fundamentals
- Schema design principles
- Advanced database concepts
- Popular systems: Access, DB2, Oracle



Access: dbbook.inf.uniroma3.it

Available for personal use. Please link to the official site in your website.

Database System Concepts

Silberschatz, Korth, and Sudarshan (7th Edition)

A definitive, comprehensive text covering fundamental concepts of database management systems. This edition provides thorough coverage of database design, languages, and implementation techniques.

Core Concepts

Foundational database principles
and architecture

Database Design

Comprehensive coverage of
modeling and schema
development

Implementation

Practical techniques for building
database systems

Perfect for: Introductory and advanced courses, self-study for professionals

Access: mrce.in/ebooks/Database System Concepts 7th Ed.pdf (International Student Edition)

Database Systems: The Complete Book

Garcia-Molina, Ullman, and Widom (Second Edition)

This second edition covers the complete spectrum of database systems material taught in university sequences. Ideal for students who want to both use and implement database systems effectively.

Key Features:

- Relational modeling fundamentals
- Comprehensive SQL coverage
- Advanced database topics
- Theoretical foundations with practical applications



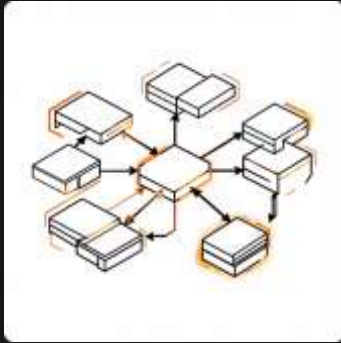
Important Note:

The PDF at people.inf.elte.hu/kiss/DB/ullman_the_complete_book.pdf is intended for instructor review and may have distribution restrictions.

Please verify usage rights before linking on your website.

Database Management Systems

Ramakrishnan and Gehrke (Second Edition)



Modern RDBMS Focus

Covers contemporary database management systems with emphasis on relational architectures



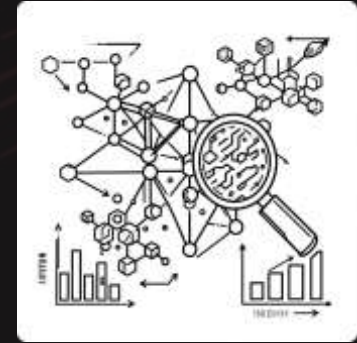
Physical Design & Tuning

Advanced topics in optimizing database performance and storage structures



Internet Databases

Integration of databases with web technologies and distributed systems



Data Mining

Techniques for extracting valuable insights from large datasets

Access: pages.cs.wisc.edu/~dbbook/openAccess/secondEdition/index.html

This link provides access to materials for the second edition, including supplementary resources.



Database Systems: Design, Implementation, and Management

Rob and Coronel

A comprehensive guide that bridges theory and practice, covering essential database concepts, practical design techniques, and real-world management strategies.



Core Concepts

Foundational database principles



SQL Mastery

Detailed coverage of SQL



Advanced Design

Transaction management



Web Integration

Modern web technologies

Access: [Direct PDF Link](#)



Curating Resources for Your Website

Strategic selection ensures your audience finds exactly what they need. Follow these best practices:

01

Identify Your Audience

Determine if your visitors are beginners learning fundamentals, intermediate users expanding skills, or advanced professionals seeking specialized knowledge

02

Select Key Texts

Choose books that align with your website's focus and complement your existing content strategy

03

Provide Clear Links

Ensure easy access with direct, working links to PDFs and verify they remain active

04

Respect Licensing

Adhere to all usage restrictions, clearly marking content for personal use only when required

Empower Your Users with Knowledge



By offering these free RDBMS books, you provide immense value to your website visitors and establish yourself as a trusted educational resource.

Start building your RDBMS library today!

Help your users master database systems and advance their careers in technology.

RDBMS Commands: Essential SQL Operations

Mastering RDBMS involves understanding key SQL commands for managing data and database structures. These commands are categorized into four main types, each serving a distinct purpose in database interaction.

1

DDL (Data Definition Language)

Commands for defining and managing database objects like tables, indexes, and views. `CREATE`, `ALTER`, `DROP` are common examples.

2

DML (Data Manipulation Language)

Used for managing data within database objects. This includes retrieving, inserting, updating, and deleting records using commands like `SELECT`, `INSERT`, `UPDATE`, and `DELETE`.

3

DCL (Data Control Language)

Manages user permissions and access control to the database. Key commands are `GRANT` (to give privileges) and `REVOKE` (to remove them).

4

TCL (Transaction Control Language)

Controls and manages transactions to ensure data integrity. Commands like `COMMIT`, `ROLLBACK`, and `SAVEPOINT` are used to manage changes.

Understanding these command types is crucial for efficient database administration and development.

TCL Commands: COMMIT, ROLLBACK, SAVEPOINT

COMMIT Example:

```
BEGIN TRANSACTION;  
INSERT INTO Students VALUES (2, 'Jane Smith', 'jane@email.com', '2024-02-01');  
COMMIT;
```

Output: Transaction committed. Changes saved permanently.

ROLLBACK Example:

```
BEGIN TRANSACTION;  
DELETE FROM Students WHERE StudentID = 2;  
ROLLBACK;
```

Output: Transaction rolled back. Changes discarded.

SAVEPOINT Example:

```
BEGIN TRANSACTION;  
INSERT INTO Students VALUES (3, 'Bob Johnson', 'bob@email.com', '2024-03-01');  
SAVEPOINT sp1;  
UPDATE Students SET GPA = 3.5 WHERE StudentID = 3;  
ROLLBACK TO sp1;  
COMMIT;
```

Output: Rolled back to savepoint sp1, then committed.

DML Commands in Action

Here's a closer look at the Data Manipulation Language (DML) commands, illustrating how they are used to interact with data within your database.

INSERT Example:

```
INSERT INTO Students VALUES (1, 'John Doe',  
'john@email.com', '2024-01-15');
```

Output: 1 row inserted.

UPDATE Example:

```
UPDATE Students SET GPA = 3.8 WHERE StudentID = 1;
```

Output: 1 row updated.

SELECT Example:

```
SELECT * FROM Students WHERE StudentID = 1;
```

StudentID	Name	Email	EnrollmentDate
1	John Doe	john@email.com	2024-01-15

DELETE Example:

```
DELETE FROM Students WHERE StudentID = 1;
```

Output: 1 row deleted.

DCL Commands: GRANT and REVOKE

Database Control Language(DCL) commands manage permissions and access rights for users on database objects.

GRANT Example:

```
GRANT SELECT, INSERT ON Students TO user_name;
```

```
Output: Privileges granted successfully.
```

GRANT with ALL Privileges:

```
GRANT ALL PRIVILEGES ON Students TO admin_user;
```

```
Output: All privileges granted to admin_user.
```

REVOKE Example:

```
REVOKE INSERT ON Students FROM user_name;
```

```
Output: INSERT privilege revoked from user_name.
```

REVOKE ALL Example:

```
REVOKE ALL PRIVILEGES ON Students FROM user_name;
```

```
Output: All privileges revoked from user_name.
```

DDL Commands: CREATE, ALTER, DROP

Data Definition Language(DDL) commands are fundamental for defining and managing the structure of your database. They allow you to create, modify, and delete database objects like tables.

CREATE TABLE Example:

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Email VARCHAR(100),  
    EnrollmentDate DATE  
);
```

Output: Table "Students" created successfully.

ALTER TABLE Example:

```
ALTER TABLE Students ADD COLUMN GPA DECIMAL(3,2);
```

Output: Column "GPA" added to Students table.

DROP TABLE Example:

```
DROP TABLE Students;
```

Output: Table "Students" dropped successfully.

Advanced Database Objects: Views, Stored Procedures, Triggers

CREATE VIEW:

```
CREATE VIEW HighAchievers AS
SELECT StudentID, Name, GPA
FROM Students
WHERE GPA >= 3.7;
```

Output: View "HighAchievers" created successfully.

Query View:

```
SELECT * FROM HighAchievers;
```

StudentID	Name	GPA
1	Alice	4.0
3	Charlie	3.8

CREATE STORED PROCEDURE:

```
CREATE PROCEDURE GetStudentGPA @StudentID INT
AS
BEGIN
    SELECT Name, GPA FROM Students WHERE StudentID = @StudentID;
END;
```

Output: Stored procedure created.

EXECUTE Stored Procedure:

```
EXEC GetStudentGPA @StudentID = 1;
```

Name	GPA
Alice	4.0

CREATE TRIGGER:

```
CREATE TRIGGER UpdateStudentTimestamp
AFTER UPDATE ON Students
FOR EACH ROW
BEGIN
    UPDATE Students SET LastModified = CURRENT_TIMESTAMP
    WHERE StudentID = NEW.StudentID;
END;
```

Output: Trigger created successfully.

DROP VIEW/PROCEDURE/TRIGGER:

```
DROP VIEW HighAchievers;
DROP PROCEDURE GetStudentGPA;
DROP TRIGGER UpdateStudentTimestamp;
```

Output: Objects dropped successfully.

Advanced TCL: Complex Transactions and Isolation Levels

Nested Transactions with Multiple Savepoints:

```
BEGIN TRANSACTION;
INSERT INTO Students VALUES (100, 'Alice', 'alice@email.com', '2024-05-09');
SAVEPOINT sp1;

INSERT INTO Enrollments VALUES (1, 100, 5);
SAVEPOINT sp2;

UPDATE Students SET GPA = 3.8 WHERE StudentID = 100;
SAVEPOINT sp3;

-- If error occurs, rollback to sp2
ROLLBACK TO sp2;

-- Continue with other operations
INSERT INTO Enrollments VALUES (2, 100, 6);
COMMIT;
```

Output: Transaction completed with selective rollback.

SET TRANSACTION Isolation Level:

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
BEGIN TRANSACTION;
SELECT * FROM Students;
COMMIT;
```

Output: Transaction executed with READ UNCOMMITTED isolation.



SERIALIZABLE Isolation:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;
SELECT * FROM Students WHERE GPA > 3.5;
UPDATE Students SET GPA = 3.9 WHERE StudentID = 1;
COMMIT;
```

Output: Transaction executed with SERIALIZABLE isolation (highest level).

Deadlock Handling:

```
BEGIN TRANSACTION;
  UPDATE Students SET GPA = 3.8 WHERE StudentID = 1;
  WAITFOR DELAY '00:00:02';
  UPDATE Courses SET Credits = 4 WHERE CourseID = 5;
COMMIT;
```

Output: Transaction completed (or rolled back if deadlock detected).

Lock Management:

```
BEGIN TRANSACTION;
SELECT * FROM Students WITH (XLOCK);
-- Exclusive lock acquired
UPDATE Students SET GPA = 3.9 WHERE StudentID = 1;
COMMIT;
```

Output: Exclusive lock released after commit.

Advanced DCL: Role-Based Access Control

Role-Based Access Control (RBAC) provides a powerful method for managing database permissions by assigning privileges to roles rather than directly to users, simplifying administration and improving security.

CREATE ROLE:

```
CREATE ROLE student_role;
CREATE ROLE instructor_role;
CREATE ROLE admin_role;
```

Output: Roles created successfully.

GRANT Privileges to Role:

```
GRANT SELECT ON Students TO student_role;
GRANT SELECT, INSERT, UPDATE ON Courses TO instructor_role;
GRANT ALL PRIVILEGES ON *.* TO admin_role;
```

Output: Privileges granted to roles.

GRANT Role to User:

```
GRANT student_role TO 'john_doe'@'localhost';
GRANT instructor_role TO 'prof_smith'@'localhost';
GRANT admin_role TO 'admin'@'localhost';
```

Output: Roles assigned to users.

REVOKE Role from User:

```
REVOKE student_role FROM 'john_doe'@'localhost';
```

Output: Role revoked from user.

CREATE USER with Password:

```
CREATE USER 'new_user'@'localhost' IDENTIFIED BY 'secure_password';
```

Output: User created successfully.

ALTER USER Privileges:

```
ALTER USER 'john_doe'@'localhost' IDENTIFIED BY 'new_password';
```

Output: User password updated.

DROP USER:

```
DROP USER 'old_user'@'localhost';
```

Output: User dropped successfully.



Advanced DML: Set Operations and CASE

Beyond basic INSERT, SELECT, UPDATE, and DELETE, advanced Data Manipulation Language (DML) features allow for more complex data retrieval and manipulation. Set operations enable combining or comparing result sets from multiple queries, while the CASE statement provides conditional logic within queries.

UNION (Combines results, removes duplicates):

```
<SQL> SELECT Name FROM Students WHERE GPA > 3.5
UNION
SELECT Name FROM Faculty WHERE Salary > 80000;
```

Output:

Name
Alice
Bob
Dr. Smith
Dr. Johnson

UNION ALL (Keeps duplicates):

```
<SQL> SELECT Name FROM CurrentStudents
UNION ALL
SELECT Name FROM AlumniStudents;
```

Output:

Name
John
Jane
John(duplicate)
Bob

INTERSECT (Common records):

```
<SQL> SELECT StudentID FROM Enrollments
WHERE CourseID = 1
INTERSECT
SELECT StudentID FROM Enrollments
WHERE CourseID = 2;
```

Output:

StudentID
5
7
12

CASE Statement:

```
SELECT Name, GPA,
CASE
  WHEN GPA >= 3.7 THEN 'Excellent'
  WHEN GPA >= 3.3 THEN 'Good'
  ELSE 'Below Average'
END as Performance
FROM Students;
```

Output:

Name|GPA|Performance
Alice|4.0|Excellent
Bob|3.5|Good
Charlie|2.9|Below Average

Advanced DML: String and Date Functions

Expanding on Data Manipulation Language (DML), these advanced functions allow for powerful data transformation and manipulation, essential for sophisticated database operations.

String Functions:

CONCAT():

```
SELECT CONCAT(FirstName, ' ', LastName) as FullName
FROM Students;
```

Output: FullName = "John Doe"

SUBSTRING():

```
SELECT SUBSTRING(Email, 1, POSITION('@' IN Email)-1) as Username
FROM Students;
```

Output: Username = "john.doe"

UPPER()and LOWER():

```
SELECT UPPER(Name) as UpperName, LOWER(Email) as LowerEmail
FROM Students;
```

Output: UpperName = "JOHN DOE", LowerEmail = "john@email.com"

LENGTH():

```
SELECT Name, LENGTH(Name) as NameLength
FROM Students WHERE LENGTH(Name) > 10;
```

Output: NameLength = 12

Date Functions:

CURRENT_DATE:

```
SELECT Name, CURRENT_DATE as Today
FROM Students;
```

Output: Today = "2024-05-09"

DATE_ADD():

```
SELECT Name, DATE_ADD(EnrollmentDate, INTERVAL 4 YEAR) as GraduationDate
FROM Students;
```

Output: GraduationDate = "2028-05-09"

DATEDIFF():

```
SELECT Name, DATEDIFF(CURRENT_DATE, EnrollmentDate) as DaysEnrolled
FROM Students;
```

Output: DaysEnrolled = 1215

EXTRACT():

```
SELECT Name, EXTRACT(YEAR FROM EnrollmentDate) as EnrollmentYear
FROM Students;
```

Output: EnrollmentYear = 2024

Advanced DML: Window Functions and CTEs

ROW_NUMBER() Window Function:

```
SELECT Name, GPA,
       ROW_NUMBER() OVER (ORDER BY GPA DESC) as Rank
FROM Students;
```

Output:

Alice	4.0	1	Bob	3.9	2	Charlie	3.8	3
-------	-----	---	-----	-----	---	---------	-----	---

RANK() and DENSE_RANK():

```
SELECT Name, GPA,
       RANK() OVER (ORDER BY GPA DESC) as Rank,
       DENSE_RANK() OVER (ORDER BY GPA DESC) as DenseRank
FROM Students;
```

Output:

Alice	4.0	1	Bob	3.9	2	Charlie	3.9	2
-------	-----	---	-----	-----	---	---------	-----	---

Common Table Expression(CTE):

```
WITH TopStudents AS (
  SELECT StudentID, Name, GPA
  FROM Students
  WHERE GPA >= 3.5
)
SELECT * FROM TopStudents
ORDER BY GPA DESC;
```

Output:

1	Alice	4.0	2	Bob	3.9	3	Charlie	3.8
---	-------	-----	---	-----	-----	---	---------	-----

PARTITION BY:

```
SELECT Name, CourseID, GPA,
       AVG(GPA) OVER (PARTITION BY CourseID) as AvgCourseGPA
FROM StudentCourses;
```

Output:

John	1	3.8	3.75	Jane	1	3.7	3.75
------	---	-----	------	------	---	-----	------

Advanced DML Commands: Joins and Aggregations

INNER JOIN:

```
SELECT s.Name, c.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
INNER JOIN Courses c ON e.CourseID = c.CourseID;
```

Output:

Name	CourseName
John Doe	Database Systems
Jane Smith	Web Development

LEFT JOIN:

```
SELECT s.Name, COUNT(e.EnrollmentID) as CourseCount
FROM Students s
LEFT JOIN Enrollments e ON s.StudentID = e.StudentID
GROUP BY s.StudentID, s.Name;
```

Output:

Name	CourseCount
John Doe	3
Jane Smith	2
Bob Johnson	0

AGGREGATE Functions:

```
SELECT COUNT(*) as TotalStudents, AVG(GPA) as AvgGPA, MAX(GPA) as MaxGPA
FROM Students;
```

TotalStudents	AvgGPA	MaxGPA
150	3.45	4.0

SUBQUERY:

```
SELECT Name FROM Students
WHERE StudentID IN (
    SELECT StudentID FROM Enrollments
    WHERE CourseID = 5
);
```

Output:

Name
John Doe
Jane Smith

Advanced DDL Commands: Constraints and Indexes

PRIMARY KEY Constraint:

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100)  
);
```

Output: Table created with StudentID as primary key.

FOREIGN KEY Constraint:

```
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)  
);
```

Output: Table created with foreign key relationship.

UNIQUE Constraint:

```
ALTER TABLE Students ADD CONSTRAINT uk_email UNIQUE (Email);
```

Output: UNIQUE constraint added to Email column.

CHECK Constraint:

```
CREATE TABLE Courses (  
    CourseID INT PRIMARY KEY,  
    Credits INT CHECK (Credits > 0 AND Credits <= 4)  
);
```

Output: Table created with CHECK constraint on Credits.

CREATE INDEX:

```
CREATE INDEX idx_student_name ON Students(Name);
```

Output: Index "idx_student_name" created successfully.

DROP INDEX:

```
DROP INDEX idx_student_name ON Students;
```

Output: Index dropped successfully.